Penetration Testing — Theory Exam Guide (20-mark edition)

ISEC6005 · essay-depth model answers for the seven theory areas · open-book reference (v4)

How to answer a 20-mark theory question

A 20-mark question rewards **structured depth**, not a brain-dump. Markers look for breadth of coverage plus evidence that you *understand*, not just recall. Use this skeleton for every long answer:

Step	What to write	Why it earns marks
1. Define	State the concept precisely in 1–2 sentences	Shows you know the term exactly
2. Explain	Unpack how/why it works, with the key components	Depth of understanding
3. Example	A concrete example — ideally from your CTF	Applies theory to practice
4. Significance	Why it matters: risk, impact, or professional consequence	Higher-order thinking
5. Link / evaluate	Connect to another area, or give pros/cons/limits	Synthesis = top band

Exam technique for 20 marks

- Budget roughly **1 mark ≈ 1–1.5 minutes**. For 20 marks, aim for 8–12 well-developed points or ~1.5–2 pages.
- **Use the mark count as a hint**: 20 marks usually = several sub-parts or one broad topic needing many angles. Cover breadth AND depth.
- Lead each paragraph with the **key term in bold**, then explain. Markers scan for keywords.
- Always include at least one **concrete example** and one **mitigation/consequence** — these separate a pass from a distinction.
- Define acronyms on first use (CIA, RoE, STRIDE, RAG). Finish with a one-sentence **synthesis** that links ideas.

1. Penetration Testing Foundations

Q: Define penetration testing and explain its purpose, characteristics, and value. (20-mark style)

Definition

Penetration testing is an **authorised, simulated cyber-attack** against a system, network, or application, performed to discover and **safely exploit** security weaknesses before a malicious attacker can. It is a controlled, legal, and methodical exercise — not random hacking.

Purpose

The purpose is **evidence-based security improvement**. A test does not merely list theoretical weaknesses; it **proves which are genuinely exploitable**, demonstrates their real-world impact, and delivers prioritised, practical remediation advice. The outcome is a measurably stronger security posture, not a pile of unverified findings.

The three defining characteristics

- **Authorisation** — explicit, documented, written permission from the system owner.

- **A defined scope** — agreed limits on what may be tested, when, and how.
- **Intent** — acting to help the organisation, never for harm or personal gain.

Remove any one of these and the *identical* technical activity becomes a criminal offence. This is the single most important idea in the unit: **professional identity, not tooling, is what makes someone a penetration tester**.

Related concepts — distinguishing the engagement types

Activity	Goal	Depth	Output
Vulnerability Assessment	Find weaknesses	Broad but shallow	List of issues
Penetration Testing	Exploit weaknesses safely	Deep	Proof of impact
Red Team	Emulate a real adversary end-to-end	Realistic	Detection & response gaps

The CIA triad — the foundation of impact

Security exists to protect three properties: **Confidentiality** (data only to authorised parties), **Integrity** (data accurate and unaltered), and **Availability** (systems usable when needed). Every vulnerability breaks one or more. Reading an exposed file breaks Confidentiality; altering a record breaks Integrity; crashing a service breaks Availability. **Always name which property a finding breaks** — it ties a technical issue to a business consequence and earns impact marks.

The phases (lifecycle)

Recon → Scanning & Enumeration → Vulnerability Analysis & Validation → Exploitation → Post-Exploitation & Privilege Escalation → Reporting. Each phase feeds the next; skipping early phases leads to wasted effort and missed findings. The **report is the real deliverable** — a test with no clear report has no value.

Methodologies & standards (name-drop for marks)

PTES (end-to-end), **NIST SP 800-115** (technical guide), **OSSTMM**, **OWASP Top 10** (web), **ISO/IEC 27001/27002**, **MITRE ATT&CK** (adversary TTPs), **CREST / PCI DSS / SANS**. Following a recognised framework makes work **repeatable, professional, legally defensible, and quality-assured** — the decisive question if something goes wrong is “did you follow an accepted methodology?”

Significance

Penetration testing matters because it converts uncertainty about security into **demonstrated, prioritised, actionable risk**. It validates defences, satisfies compliance, and — done professionally — proves risk in the safest, most defensible way possible.

Link to your CTF

- Black-box test; nmap = active recon; curl/smbclient/ftp = enumeration; reading a flag = demonstrating impact (which CIA property?); your write-up = the reporting phase.

2. Ethics and Law

Q: Explain the legal and ethical framework that makes penetration testing legitimate. (20-mark style)

The core principle

Penetration testing is legal **only because of ethics, authorisation, and professional conduct**. A tester and a criminal perform the *same keystrokes*; the only difference is **authorisation, scope, and intent**. “Good intentions” are not a legal defence — scanning a network without permission is a criminal offence even if you find and report a vulnerability.

The three legal prerequisites

- **Written authorisation** — explicit, documented permission from the owner.
- **A defined scope** — testing outside scope is illegal activity, even accidentally.
- **Rules of Engagement (RoE)** — the formal agreement on how the test is conducted.

The Australian legal landscape

Central legislation: the **Cybercrime Act 2001** and the **Privacy Act 1988**, with the **Criminal Code Act 1995** underpinning the computer offences. Offence categories: **unauthorised access** to data, **unauthorised alteration** of data, **interception of communications**, and **privacy violations**. Equivalent overseas laws: UK **Computer Misuse Act 1990**, US **Computer Fraud and Abuse Act (CFAA)**.

Consequences of unauthorised / out-of-scope testing

- **Personal liability** — legal, disciplinary, or criminal consequences for the individual.
- **Organisational liability** — regulatory, contractual, financial exposure for the firm.
- **Reputational damage** — lost trust and career standing. *Most cybersecurity careers end not from poor skill but poor judgement.*

Rules of Engagement — what a complete RoE defines

Section	What it defines
Scope / Out of scope	Systems allowed, and what must never be touched
Testing methods allowed	Scanning, exploitation, social engineering, etc.
Testing window	Dates/times testing may occur
Authorised personnel	Who may perform the test
Data-handling rules	How sensitive data must be treated
Communication plan	Who to contact if something goes wrong
Stop conditions	When testing must immediately stop
Liability & consent	Written authorisation from the client

Ethical codes and principles

Testers are bound by the **ACS Code of Ethics** (Australia), **EC-Council**, and **ISC²**. Shared principles: **protect society / public interest**, **authorisation & legality**, **competence**, **honesty**, **confidentiality**, **professionalism**, and **responsible disclosure** (report privately to the owner first, allow time to patch before any public disclosure).

Why ethics matters most in offensive security

Testers are granted powerful, trusted access to systems and sensitive data. The ability to find and exploit weaknesses can cause serious harm if misused. The only thing separating a professional from a criminal is authorisation and intent. An ethical hacker is defined by what they choose **not** to do — the guiding question is always “just because you *can*, *should* you?”

3. Threat Modeling & Attack Surface Analysis

Q: Explain threat modeling and attack surface analysis, including STRIDE, PASTA and ATT&CK.; (20-mark style)

Definition and purpose

Threat modeling is the **structured process of identifying what can go wrong, who might cause it, and how** — risk-based thinking, not guessing. It is done **before** tools so testing is targeted rather than blind: scanning without understanding wastes time, overlooks key vulnerabilities, risks disrupting production, and

tests irrelevant components. It answers four questions: **What are we protecting?** (assets), **Who from?** (threat actors), **How could they attack?** (attack paths), **What is the impact?** (business risk).

The process

Identify assets → map the system (User → Web → App → Database) → find trust boundaries → analyse risks (Impact × Likelihood) → prioritise & mitigate highest risk first.

STRIDE — application-level threat categories

Letter	Threat	Example	Mitigation
S	Spoofing	Stolen credentials / spoofed IP-DNS	MFA, strong auth
T	Tampering	Modifying DB records / packets	Integrity checks, signing
R	Repudiation	Denying action, no audit trail	Logging & monitoring
I	Information Disclosure	Unencrypted data, exposed files	Access control, encryption
D	Denial of Service	Flooding, crashing a service	Rate limiting, WAF
E	Elevation of Privilege	User gains root/admin	Least privilege, patching

Comparing the three frameworks

Framework	Focus	Perspective	Output
STRIDE	Threat types per component	Defensive	What can go wrong
PASTA	Risk & business impact (7 stages)	Attacker + business	Prioritised risks
MITRE ATT&CK	Real adversary behaviour (TTPs)	Attacker	How attacks happen

Assets, trust boundaries, and why attacks concentrate there

Assets = things worth protecting (PII, financial records, credentials, tokens, APIs, payment systems). A **trust boundary** is where trust/control/privilege changes (Internet → Web server; Web → Database; Internal → Admin portal). **Attacks happen at trust boundaries** because that is where untrusted input meets trusted processing. STRIDE identifies what can go wrong at each boundary, PASTA assesses the business risk, and ATT&CK shows how attackers cross them.

Attack surface — and why size matters

The **attack surface** is all possible entry points. **External** = internet-facing (public IPs, web apps, APIs, email). **Internal** = reachable after compromise, enabling lateral movement (Active Directory, file shares, internal services, misconfigurations). **Risk is about exposure, not just severity** — a minor flaw on a large surface can be high risk because there are more opportunities to succeed. Reduce the surface: close unused ports, disable services, remove anonymous access, segment networks.

How threat modeling translates into testing

It turns random vulnerability discovery into **risk-driven validation**. Two techniques: (1) **map business objectives to attacker goals** (“protect customer data” → “exfiltrate the customer DB” → “test all feasible data-extraction paths”); (2) **convert the model into testable hypotheses** (“admin portal reachable + weak password policy” → “attacker may gain admin via password spraying”). Every test then has clear justification and traceability from risk to execution.

4. Vulnerability Analysis & Validation

Q: Explain vulnerability analysis, validation, and risk-based prioritisation. (20-mark style)

Definition

Vulnerability analysis is **identifying, verifying, and assessing weaknesses**. The key realisation: after enumeration, **not every vulnerability is real, exploitable, or relevant**. If a scanner reports 100 issues you do not blindly exploit all 100 — you confirm which represent genuine, exploitable risk, then prioritise. Common sources: outdated software, misconfigurations, weak authentication, exposed services, poor access controls.

Why validation is essential

Automated scanners are fast but **produce false positives, incomplete results, and misleading severity**. Professionals never rely solely on them. Validation confirms three things: the vulnerability **actually exists** (not a scanner artefact), it **behaves as expected** (genuinely exploitable in this environment), and it is **in scope**. This improves the accuracy and credibility of the whole assessment.

Validation methods

- **Manual verification** — hands-on confirmation rather than trusting the scanner.
- **Banner grabbing / version checking** — confirm the exact software and that the running version is the vulnerable one.
- **Controlled, non-destructive testing** — safe, read-only requests that test exploitability without damage.
- **Configuration review** — inspect settings that introduce weakness. (Scans may be unauthenticated or authenticated.)

True / False Positives and Negatives

Term	Meaning	Why it matters
True Positive	Real vuln, correctly detected	Accurate reporting
True Negative	No vuln, none reported	Trustworthy results
False Positive	Reported but does NOT exist	Wasted time, lost credibility
False Negative	Real vuln but MISSED	Most dangerous — false security

Risk-based prioritisation

Priority is driven by **Risk = Likelihood × Impact**. Likelihood depends on ease of exploitation, access level required, and exposure (external vs internal); Impact depends on data sensitivity, system criticality, and business disruption. A **CVE** is a public identifier; **CVSS** is a 0–10 technical-severity score — but **priority must be based on risk in context, not severity alone**. A CVSS 9–10 can be *lower* priority if hard to exploit, needs high privileges, or has little business impact; a medium can be *high* if actively exploited or on a critical system.

Context and chained vulnerabilities

The same flaw carries different risk by context: public-facing = high, internal/isolated = lower. **Chained vulnerabilities** matter most — several individually low-risk issues can combine into a high-risk path (information leak + weak password = full account compromise). *A professional is not the one who finds the most vulnerabilities, but the one who correctly identifies the most important ones.*

Test-hypothesis thinking

Professionals do not randomly attempt exploits; they develop **test hypotheses** from validated evidence. Validation tells you what is real; prioritisation tells you what to exploit first; the hypothesis makes exploitation **targeted, efficient, and defensible**.

5. Exploitation Fundamentals

Q: Explain exploitation, its classes, reverse shells, privilege escalation, and post-exploitation. (20-mark style)

Definition and position in the lifecycle

Exploitation is **leveraging a vulnerability to make a system behave in an unintended way** — executing unauthorised actions, gaining privileges, accessing protected data, or disrupting availability. It sits after validation and before post-exploitation, and should be **evidence-driven**: each action follows logically from a validated finding, never random trial-and-error.

Key distinctions

Term	Meaning
Vulnerability	The underlying weakness
Exploit	The technique/code that abuses it
Payload	What runs once the exploit works (e.g. a reverse shell)
Post-exploitation	Actions after access: privesc, credential access, lateral movement, persistence, impact

Most exploits have a **trigger** (activates the vulnerability), an **execution mechanism** (memory misuse, logic bypass, trust-boundary violation), a **payload** (code execution, command execution, data disclosure, disruption), and **constraints** (OS, architecture, ASLR/DEP/permissions).

Classes of exploitation

- **Memory-related** — buffer overflows, use-after-free; countered by compiler protections and hardening.
- **Web application** — injection, broken auth/access control; countered by input validation, secure design, least privilege.
- **Authentication & authorisation abuse** — insecure role checks, session-handling flaws (exploiting logic, not implementation).
- **Configuration-based** — insecure defaults, misconfigurations, default credentials, excessive permissions.

Key insight: many modern breaches exploit mistakes in design, business logic, or trust assumptions — not just "broken code." Exploits may run manually or via **Metasploit**; ready-made code via **searchsploit / Exploit-DB** — but must always match a *validated* vulnerability.

Reverse shell vs bind shell

A **reverse shell** has the **target initiate an outbound connection back to the attacker**, who is listening (`nc -lvnp 4444`). It is preferred over a **bind shell** (attacker connects inbound to a port the target opens) because **firewalls typically block inbound but allow outbound** — so the target "calling home" slips through.

Privilege escalation — common Linux vectors

Moving from a low-privileged foothold to higher privileges (ideally root). Real impact occurs after privesc. Either **user-space** (abusing misconfigurations; low-noise, common) or **kernel** (OS bugs; high-risk, last resort). Vectors:

- **SUID binaries** (GTFOBins): `find / -perm -u=s -type f 2>/dev/null`
- **sudo misconfiguration**: `sudo -l` — a NOPASSWD interactive binary gives root.
- **Cron job abuse**, **writable/hijackable PATH**, **kernel exploits** (`uname -r` + searchsploit), **credential harvesting** (SSH keys, configs, .bash_history).

Post-exploitation and why it matters

Everything after initial access: privesc, credential access, lateral movement, persistence, data access. The initial exploit merely opens the door — **post-exploitation determines the real business impact**, and most damage occurs here. For defenders: assume initial exploitation can succeed, and invest in **segmentation, detection, containment, and response**. Understanding *how* an exploit works (attacker view) directly informs *how to detect and prevent it* (defender view). Ethical exploitation demonstrates risk **without causing harm** —

no out-of-scope actions, no destructive payloads, minimum necessary testing.

6. Mitigation Strategies

Q: For common findings, explain cause and mitigation, and the defence-in-depth principles. (20-mark style)

For every finding, explain the **cause**, the **trust assumption violated**, the **blast radius**, and the **business impact** — then how to **validate** and **mitigate**. Mitigation removes the cause and reduces exposure.

Vulnerability	Cause	Mitigation
Anonymous FTP	Auth not required	Disable anon login; SFTP/FTPS; remove sensitive files
Anonymous / open SMB	Guest/null session allowed	Disable guest; enforce auth; least-privilege shares
Directory listing	Indexing enabled	Disable browsing (Options -Indexes); no obscurity
Information disclosure	Sensitive files/IDs exposed	Access control; non-guessable IDs; object-level authz
SQL / command injection	Input concatenated into query/shell	Parameterised queries; validation; least-priv DB; never pass input to shell
Weak / default creds	No policy / defaults unchanged	Strong policy; change defaults; MFA; lockout; salted hashing
Known-CVE / outdated	Unpatched software	Patch promptly; asset/patch inventory
Privilege escalation	Misconfig / excess privilege	Least privilege; fix sudo/SUID; correct permissions; patch

Defence-in-depth principles

- **Least privilege** — users/services get only the access they need.
- **Defence in depth** — multiple overlapping controls so one failure is not total.
- **Reduce attack surface**, **patch management**, **network segmentation**, **logging & monitoring** (counters Repudiation), **strong authentication** + MFA.

The universal finding framework (use for every flag)

- **Cause** → **trust assumption violated** → **blast radius** → **business impact** → **mitigation.** This is applied threat modeling + vulnerability analysis, and finishing with a one-line mitigation reliably earns extra marks.

7. Emerging Trends: AI & Autonomous Agents in Offensive Security

The **new** theory area, and a strong candidate for a 20-mark question. It maps onto the **OWASP Top 10 for LLM Applications**. The thesis to open and close with: **agent risk grows when language-based reasoning is connected to real permissions and actions** — and the classic fundamentals (sections 1–6) still apply to this new surface.

Core concepts (define these first)

- **LLM** — generates/interprets content from learned patterns and context.
- **AI agent** — uses a model in a **control loop** to pursue a goal.
- **Tool calling** — lets the model request an operation from an external component (API, DB, shell).
- **RAG** — Retrieval-Augmented Generation; supplies retrieved external information as model context.
- **Guardrails** — policies/mechanisms intended to constrain unsafe behaviour (but not a substitute for access control).

Prompt injection (the headline risk)

Direct injection: a user tries to override the system's intended task. **Indirect injection**: malicious instructions are embedded in a document, email, or webpage the agent retrieves, so it may **treat untrusted content as instructions**. **Why prompts alone fail:** instructions in a prompt cannot enforce data permissions or tool-level authorisation — they are suggestions, not controls. Test with benign, controlled payloads and observe whether the workflow changes. **Mitigate** via trusted/untrusted data separation, backend controls, and restricted tools. (Think of it as the SQL injection of the AI era: untrusted input treated as a command.)

Sensitive information disclosure

Excessive retrieval permissions may expose records beyond the requester's authority; **shared service identities** blur user vs agent rights; conversation history/memory may **leak across users or sessions**; sensitive data may appear in prompts, traces, logs, or third-party services. **Control:** validate access **at the data source** before information reaches the model; use synthetic data and verify both allowed and denied paths.

Excessive agency and tool abuse

Three questions: **Functionality** — tools the agent does not need? **Permissions** — can its identity access/modify too much? **Autonomy** — consequential actions without approval? Example: a support agent that can *delete* records when it only needs to *create tickets*. **Design for bounded autonomy**, transaction limits, and human approval for high-impact actions, to limit the impact of model error or manipulation.

Insecure output handling

Generated output is untrusted when passed downstream. Unsafe insertion into web pages → XSS; generated text in queries/commands → injection or unintended execution. **Controls:** parameterised operations, context-appropriate encoding, schema validation, allowlisted actions. *Traditional application-security fundamentals still apply.*

Memory, RAG, and multi-agent risks

- **Memory** — can one user's session data reach another? **RAG** — retrieved docs crossing boundaries or attacker-influenced? **Multi-agent** — one agent passing malicious instructions to another? **Third-party** — a plugin/dependency compromising a workflow? **Assess data flow, identity propagation, and trust boundaries end to end.**

AI-assisted penetration testing — uses and the critical limit

Uses: **recon** (summarise authorised inventories), **planning** (hypotheses from architecture/threat models), **code review** (suspicious patterns, test cases), **analysis** (correlate scans, explain impact), **reporting** (draft —

then verify). **The critical rule: AI output is a hypothesis, not proof of exploitability** — validate manually, exactly like a scanner result. *More automation does not automatically mean better assurance.*

Approach	What it does	Strength / limit
Conventional automation	Repeatable checks vs known patterns	Fast but shallow
AI-assisted testing	Interprets context, proposes hypotheses	Needs human validation
Agentic testing	Plans/executes multi-step actions with tools	Powerful but higher risk
Human-led assessment	Business context, logic, consequence	Best judgement; slower

Responsible testing workflow (6 steps)

1. Define written scope, authority, test accounts, data, prohibited actions. 2. Establish a reliable baseline (architecture, identities, permissions, logs). 3. Generate **evidence-grounded** hypotheses. 4. Validate safely with minimal-impact, approved tests. 5. **Distinguish demonstrated access from theoretical impact.** 6. Report evidence, limitations, remediation, retest. Preserve reproducibility; stop if testing exceeds authorised boundaries.

Defence-in-depth technical controls for AI systems

- Enforce **server-side authorisation** for every data request and action; **least privilege** for service identities and tools; **separate trusted instructions from untrusted retrieved content**; validate inputs/outputs with schemas and allowlists; isolate sessions and enforce access control during retrieval; require **human approval** for high-impact or irreversible operations.

Monitoring, governance, accountability

Log identity context, tool calls, approvals, denied actions, outcomes; protect logs (they hold sensitive data); run adversarial tests and regression evaluations after changes; govern third-party models/plugins/dependencies; **assign clear accountability — do not delegate professional responsibility to a model**.

Worked case study (model structure for a scenario question)

Scenario: an agent retrieves a confidential student-support document after following untrusted instructions embedded in a knowledge-base article; its service identity had broader access than the staff member; logs show an API request but not the initiating user.

- **Causes** — indirect prompt injection, weak authorisation boundaries, excessive permissions, incomplete audit attribution.
- **Evidence** — controlled input, test identity, redacted API records, timestamps, synthetic data-access outcome.
- **Remediation** — enforce user-level backend authorisation; reduce service permissions; add untrusted-content handling, approval controls, attributable logging.
- **Retest** — original and equivalent cases; verify denial behaviour and audit completeness. *Separate demonstrated impact from plausible but unverified consequences.*

Discussion: should agents act autonomously?

Weigh where autonomy suits **low-risk, reversible** tasks; when human approval must be mandatory; how permissions, transaction limits, rollback, and monitoring interact; and who is accountable. **A defensible policy uses bounded autonomy and stronger controls for sensitive actions.**

How to score top marks on an AI question

- Open and close with the thesis: **risk grows when reasoning connects to real permissions/actions**.
- Name the OWASP-style risks (prompt injection, sensitive disclosure, excessive agency, insecure output) and **give a one-line control for each**.
- Make the synthesis point: **the classic fundamentals still apply** — injection, output encoding, least privilege, access control — AI just adds a new untrusted-input surface.
- Always separate **demonstrated access** from **theoretical impact**, and keep **human accountability** — never delegate responsibility to a model.

The seven areas connect into one story: you threat-model to plan, enumerate the attack surface, analyse and validate vulnerabilities, exploit ethically within scope, recommend mitigations, and report — applying the same discipline to AI systems as a new untrusted-input surface. Every CTF flag is a live example of cause / trust / blast radius / impact / mitigation.

ISEC6005 — Theory Exam Guide v4 (20-mark edition). Open-book revision reference. Generative-AI tools are not permitted during the exam itself.